

	PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINHAS GERAIS Instituto de Ciências Exatas e Informática (ICEI) Departamento de Sistemas de Informação e Engenharia de Software Graduação em Engenharia de Software RELATÓRIO TÉCNICO – MÉTODO ATAM <i>(Architecture Tradeoff Analysis Method)</i>	
	PROFESSOR: FILIPE TÓRIO LOPES RUAS NHIMI	
	DISCIPLINA: ARQUITETURA DE SOFTWARE	PERÍODO: 5º
	PROJETO: Pet-Trail	DATA: 28/06/2026

1. Resumo Executivo

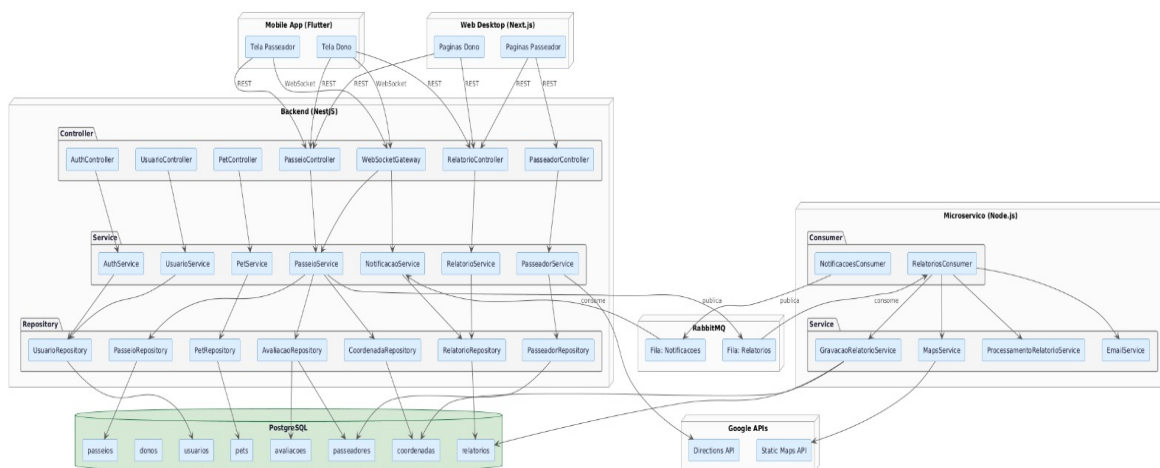
O PetTrail é uma plataforma mobile e web que conecta tutores a passeadores autônomos, com rastreamento GPS em tempo real, confirmação presencial por código e geração automática de relatórios. A análise ATAM foi demandada pela informalidade do mercado de passeio de pets sem rastreabilidade nem transparência para o tutor e pela necessidade de validar se a arquitetura proposta sustenta confiança, tempo real e comprovação do serviço. Ao final do processo, o grupo concluiu que a solução, baseada em NestJS, Flutter, Next.js, PostgreSQL, RabbitMQ, Firebase e Google Maps, atende aos direcionadores críticos do MVP, com trade-offs aceitos em custo operacional e dependência de serviços externos.

2. Objetivos de Negócio e Direcionadores Arquiteturais

- **Direcionador 1: Rastreamento Transparente em Tempo Real** — Replicação imediata das coordenadas GPS capturadas pelo passeador para o mapa do tutor, com trilha crescente durante todo o passeio.
- **Direcionador 2: Comprovação e Integridade do Serviço** — Geração automatizada de relatórios pós-passeio (trajeto, distância, duração e imagem do percurso) sem perda de integridade dos dados registrados
- **Direcionador 3: Confiança na Retirada Presencial** — Validação obrigatória por código de confirmação antes do início do rastreamento, garantindo que o passeador correto retirou o pet no local.
- **Direcionador 4: Resiliência no Encerramento de Passeios** — Processamento assíncrono de relatórios via fila, absorvendo picos de carga sem bloquear a finalização do passeio pelo passeador.

- Direcionador 5: Segurança e Controle de Acesso por Perfil — Autenticação JWT em todos os endpoints, com segregação de permissões entre tutor e passeador e credenciais isoladas em variáveis de ambiente.
- Direcionador 6: Viabilidade Operacional do MVP — Operação sustentável dentro dos limites de free tier da AWS e do crédito mensal da Google Maps Platform, sem comprometer as funcionalidades essenciais.

3. Topologia e Abordagem da Arquitetura



Componente / Camada

Tecnologia Proposta

Papel Estrutural e Mecanismo de Comunicação

Canais de Apresentação

Flutter e Next.js

Representam as interfaces de acesso ao sistema, separadas entre aplicativo mobile e aplicação web. O mobile atende fluxos de passageiro e dono, enquanto o desktop web atende páginas de dono e passeador. A comunicação com o backend ocorre principalmente por REST, com uso de WebSocket em fluxos que exigem atualização em tempo real.

Back-End Modular

NestJS

Camada central da aplicação, organizada internamente em controllers, services e repositories. Expõe endpoints REST, gerencia comunicação WebSocket e concentra as regras de negócio dos módulos de autenticação, usuários, pets, passeios, notificações, relatórios e passeadores.

Persistência Relacional	PostgreSQL	Base de dados principal do sistema. Armazena as entidades persistentes utilizadas pelo backend, incluindo passeios, donos, usuários, pets, avaliações, passeadores, coordenadas e relatórios. A comunicação ocorre por meio dos repositories do backend.
Mensageria Assíncrona	RabbitMQ	Barramento responsável por desacoplar tarefas que não precisam ser executadas diretamente no fluxo síncrono da requisição. O backend publica mensagens nas filas de notificações e relatórios, que posteriormente são consumidas pelo microserviço.
Microserviço de Processamento	Node.js	Componente independente responsável pelo processamento assíncrono de notificações e relatórios. Possui consumers para as filas RabbitMQ e serviços internos para gravação de relatórios, processamento de relatórios, integração com mapas e envio de e-mails.
Integrações Externas	Google APIs e Serviço de E-mail	Serviços externos utilizados pelo sistema para funcionalidades complementares. As APIs do Google apoiam recursos de rotas e mapas, enquanto o serviço de e-mail viabiliza o envio de notificações e comunicações geradas pelo microserviço.

4. Árvore de Utilidade e Inventário de Cenários

Insira os cenários detalhados construídos para estressar os atributos de qualidade. Lembre-se de estruturar cada cenário contendo: Estímulo, Ambiente e Resposta esperada.

ID	Atributo de Qualidade	Descrição do Cenário (Estímulo, Ambiente e Resposta)	Prioridade (Negócio / Dificuldade)
C-01	Performance	Estímulo: durante um passeio ativo, o app Flutter do passeador captura coordenadas GPS a cada 5 segundos e as envia via WebSocket ao Backend NestJS. Ambiente: sistema em operação normal, com passeio no status Em Passeio e tutor conectado ao mapa. Resposta: o Backend deve persistir a coordenada e replicá-la para o mapa do tutor em no máximo 30 segundos após a captura, mantendo a trilha crescente visível em tempo real.	Alta / Alta
C-02	Disponibilidade	Estímulo: o passeador inicia o rastreamento e bloqueia a tela ou coloca o app Flutter em segundo plano durante todo o passeio. Ambiente: Android ou iOS com flutter_background_service ativo e passeio em andamento. Resposta: o app deve continuar capturando e transmitindo coordenadas GPS sem interrupção, sem perda de pontos por encerramento do processo em foreground.	Alta / Alta

C-03	Disponibilidade	Estímulo: múltiplos tutores e passeadores realizam passeios simultâneos, mantendo conexões WebSocket abertas para notificações e rastreamento. Ambiente: backend NestJS e gateway WebSocket em operação normal sob carga concorrente de sessões ativas. Resposta: o sistema deve manter as conexões estáveis e o fluxo de solicitação, aceite e acompanhamento de passeios sem queda generalizada de sessões em andamento.	Alta / Alta
C-04	Disponibilidade	Estímulo: o passeador finaliza um passeio pelo app Flutter enquanto o Microserviço de Relatórios (Node.js) está indisponível ou sob alta latência. Ambiente: fila Relatorios do RabbitMQ operacional; worker offline ou degradado. Resposta: o Backend deve aceitar a finalização do passeio imediatamente, publicar o payload na fila e reter a mensagem com segurança até o processamento, sem bloquear o encerramento pelo passeador.	Alta / Média

C-05	Segurança	Estímulo: o passeador tenta iniciar o rastreamento informando código de confirmação inválido, expirado ou ausente. Ambiente: passeio no status Passeador a Caminho, com código gerado pelo Backend e entregue ao tutor via WebSocket. Resposta: o Backend deve rejeitar a requisição, impedir a transição para Em Passeio e bloquear o início da transmissão GPS até a validação correta do código presencial.	Alta / Média
C-06	Segurança	Estímulo: requisição REST a endpoint restrito sem token JWT válido ou com perfil incorreto (ex.: tutor acessando rota exclusiva de passeador). Ambiente: API NestJS em operação normal com Guards de autenticação ativos. Resposta: o sistema deve retornar erro de autorização antes de executar a lógica de negócio, garantindo acesso apenas aos recursos condizentes com o perfil embutido no token.	Alta / Média

C-07

Escalabilidade

Estímulo: vários passeios são finalizados em intervalo curto, gerando picos de publicação na fila Relatorios. Ambiente: Backend NestJS, RabbitMQ (Amazon MQ) e um ou mais workers Node.js consumindo a fila. Resposta: o Backend deve permanecer responsivo às requisições síncronas enquanto o RabbitMQ absorve a volumetria e os workers processam relatórios de forma assíncrona, sem degradação perceptível no fluxo principal.

Alta / Baixa

C-08

Modificabilidade

Estímulo: necessidade de substituir ou incluir um novo canal de notificação pós-passeio (ex.: push notification além de e-mail e pop-up WebSocket). Ambiente: arquitetura com fila Notificacoes e NotificacoesConsumer no Microserviço isolado do fluxo mobile. Resposta: a alteração deve ser implementada exclusivamente na camada do worker e do consumer de notificações, sem modificar o fluxo de solicitação, rastreamento ou finalização de passeio no Backend ou no app Flutter.

Média / Baixa

5. Inventário Analítico Rastreável (Saídas do ATAM)

Classifique e detalhe os achados arquiteturais mapeados a partir do cruzamento dos cenários com a topologia:

Riscos Identificados (R)

- R-01: Ponto único de falha no broker de mensageria
Cenários afetados: C-04, C-07 | Componente crítico: RabbitMQ (Amazon MQ instância single-instance)
Descrição técnica: A arquitetura concentra o desacoplamento assíncrono em um único broker RabbitMQ. Se o Amazon MQ ficar indisponível no momento da finalização de passeios, o Backend NestJS não consegue publicar payloads na fila Relatorios, impedindo a geração de relatórios e quebrando o fluxo pós-passeio — mesmo que o encerramento síncrono ainda seja aceito quando a fila já estava operacional. No MVP, a instância única amplifica esse risco operacional frente a picos de finalização simultânea.
- R-02: Dependência crítica de APIs externas do Google Maps
Cenários afetados: C-01, C-04, C-07 | Componente crítico: Google Directions API / Static Maps API
Descrição técnica: O cálculo de ETA na aceitação do passeio (Directions API) e a geração da imagem PNG do trajeto no worker (Static Maps API) dependem integralmente de serviços externos com quota e latência fora do controle da aplicação. Indisponibilidade, throttling ou estouro do crédito mensal (RNF010) degradam a experiência em tempo real e retêm mensagens na fila Relatorios, acumulando backlog no RabbitMQ sem garantia de SLA contratual.
- R-03: Concentração de conexões WebSocket no Backend unificado
Cenários afetados: C-01, C-03 | Componente crítico: WebSocketGateway (NestJS — Render)
Descrição técnica: Toda transmissão de coordenadas GPS, notificações de solicitação/aceite e atualização de status convergem para o mesmo processo NestJS hospedado em instância única no Render. Sob múltiplos passeios simultâneos, o gateway WebSocket torna-se gargalo de CPU e memória; queda ou restart desse processo interrompe todas as sessões ativas de rastreamento, impactando diretamente tutores em passeios em andamento.

- R-04: Fragilidade do rastreamento GPS em background no mobile
Cenários afetados: C-02 | Componente crítico: Flutter App (geolocator + flutter_background_service)
Descrição técnica: A captura contínua de GPS em segundo plano depende de políticas distintas entre Android e iOS (limitações de execução em background, permissões de localização e otimização de bateria). Mesmo com flutter_background_service, o SO pode suspender ou reduzir a frequência de atualizações, gerando lacunas na trilha e violando o requisito de rastreamento percebido como tempo real (RNF001).

Pontos de Não-Risco (NR)

- NR-01: Desacoplamento assíncrono na finalização de passeios
Cenários afetados: C-04, C-07
Descrição técnica: A publicação do payload na fila Relatorios pelo Backend NestJS, consumida pelo Microserviço Node.js, anula a fragilidade de bloquear o passeador na geração de relatório. Mesmo com o worker offline ou degradado, a finalização síncrona é concluída imediatamente e a mensagem permanece retida no RabbitMQ até reprocessamento táctica de message queue que mitiga falha parcial sem perda do encerramento operacional.
- NR-02: Validação presencial por código de confirmação
Cenários afetados: C-05
Descrição técnica: A exigência de código gerado server-side e validado pelo PasseioService antes da transição para Em Passeio impede início fraudulento do rastreamento. A decisão de centralizar a validação no Backend (e não no client Flutter) anula o risco de bypass via manipulação local do app, garantindo que nenhuma coordenada GPS seja aceita sem confirmação presencial verificada.
- NR-03: Autenticação stateless com controle de perfil em JWT
Cenários afetados: C-06
Descrição técnica: O uso de NestJS Guards com perfil (tutor/passeador) embutido no token JWT elimina a necessidade de consulta ao banco a cada requisição para autorização e impede acesso cruzado entre rotas restritas. A tática de Role-Based Access Control stateless mitiga escalonamento horizontal de permissões sem acoplamento adicional à camada de persistência.
- NR-04: Isolamento de notificações na camada do worker
Cenários afetados: C-08
Descrição técnica: A separação do NotificacoesConsumer no Microserviço Node.js, alimentado pela fila Notificacoes, permite evolução de canais (e-mail, pop-up WebSocket, push) sem alterar o fluxo mobile ou os Controllers do Backend. A tática de separation of concerns por fila dedicada anula o risco de regressão no núcleo transacional ao incluir novos canais de comunicação.

Tradeoffs Técnicos (T)

- T-01: Latência de rastreamento em tempo real vs. Custo operacional do MVP
Cenários afetados: C-01, C-03
Descrição técnica: Rastreamento percebido como tempo real exige WebSocket persistente e transmissão a cada 5 segundos, elevando consumo de conexões, CPU e quota de APIs de mapas. Para manter viabilidade dentro do free tier (RNF010), optou-se por intervalo de captura de 5s e tolerância de latência de até 30s (RNF001), sacrificando precisão sub-segundo em favor de sustentabilidade financeira do MVP.
- T-02: Consistência imediata vs. Processamento assíncrono de relatórios
Cenários afetados: C-04, C-07
Descrição técnica: A geração síncrona de relatório (cálculo Haversine, Static Maps PNG, e-mail, gravação) bloquearia o passeador por vários segundos a cada finalização. A escolha de engenharia foi publicar na fila Relatorios e retornar imediatamente ao app aceitando consistência eventual, o tutor recebe o pop-up e o relatório com atraso tolerável em troca de disponibilidade e responsividade do fluxo principal.
- T-03: Backend monolítico unificado vs. Microserviços completos
Cenários afetados: C-03, C-07, C-08
Descrição técnica: Concentrar autenticação, lógica de passeio, WebSocket e persistência em um único Backend NestJS simplifica desenvolvimento e deploy no prazo acadêmico, mas impede escalar independentemente o gateway WebSocket do processamento REST. Apenas o worker de relatórios foi extraído como microserviço, trade-off consciente entre modificabilidade/velocidade de entrega e escalabilidade horizontal granular.
- T-04: Dependência de Google Maps vs. Autonomia de infraestrutura
Cenários afetados: C-01, C-04, C-07
Descrição técnica: Utilizar exclusivamente Google Maps Platform (restrição arquitetural) acelera integração de mapas, rotas e imagens estáticas, mas cria acoplamento a vendor externo com custo variável e indisponibilidade possível. A alternativa (tile server próprio + engine de rotas) ofereceria controle total, porém violaria restrições do projeto e aumentaria complexidade operacional, escolheu-se time-to-market e qualidade cartográfica em detrimento de soberania e previsibilidade de custo

Pontos de Sensibilidade (S)

- S-01: Intervalo de captura GPS e limiar de latência de replicação
Descrição técnica: O requisito RNF001 fixa captura a cada 5 segundos e replicação em até 30 segundos. Reduzir o intervalo para 1–2 segundos (melhorando C-01) multiplica volume de mensagens WebSocket, escritas no PostgreSQL/Firebase e consumo de bateria mobile (C-02), podendo saturar o WebSocketGateway (C-03) sem upgrade de infraestrutura. Pequena alteração nesse parâmetro impacta fortemente Performance, Disponibilidade e Custo operacional.
- S-02: Disponibilidade e configuração do RabbitMQ (Amazon MQ)
Descrição técnica: Toda finalização de passeio e encadeamento de notificações depende da publicação bem-sucedida na fila Relatorios. Alterar timeout de conexão, política de retry ou migrar de single-instance para cluster altera minimamente a configuração, mas impacta fortemente Disponibilidade (C-04) e Escalabilidade (C-07), fila indisponível ou mal configurada interrompe todo o pipeline assíncrono pós-passeio
- S-03: Capacidade de conexões simultâneas do WebSocketGateway NestJS
Descrição técnica: Cada passeio ativo mantém pelo menos duas conexões WebSocket (passeador transmitindo + tutor recebendo), além de conexões para notificações de solicitação/aceite. Aumentar o número de passeios concorrentes sem escalar horizontalmente o Backend ou implementar sticky sessions/load balancer para WebSocket causa queda de sessões e perda de coordenadas, impacto direto e desproporcional em Performance (C-01) e Disponibilidade (C-03).
- S-04: Quota e latência das Google Maps APIs
Descrição técnica: O worker invoca Static Maps API para cada relatório gerado e o Backend consome Directions API a cada aceite de passeio. Pequena variação na quota diária, no plano de billing ou na latência média da API (ex.: throttling após pico de finalizações) retém mensagens na fila Relatorios e atrasa entrega de comprovação ao tutor, impacto forte em Disponibilidade (C-04), Escalabilidade (C-07) e Integridade do serviço sem qualquer mudança interna no código da aplicação.

6. Plano de Ação e Recomendações Técnicas

Com base nos riscos R-01 a R-04 e nos trade-offs T-01 a T-04 identificados no inventário analítico, recomenda-se o plano abaixo para evolução da arquitetura do PetTrail, priorizando mitigação de falhas que impactam passeios ativos e o pipeline pós-passeio.

- Ação de Curto Prazo (Mitigação Crítica): Configurar dead-letter queues (DLQ) e política de retry com backoff exponencial nas filas Relatorios e Notificacoes do RabbitMQ, garantindo retenção segura de payloads quando o worker estiver offline ou a Static Maps API falhar (R-01, R-02, C-04). Implementar health checks e alertas de indisponibilidade no Backend NestJS, worker Node.js e Amazon MQ via Render/AWS CloudWatch, com notificação imediata em queda de serviço durante horário de operação (R-01, R-03).
- Ação de Curto Prazo (Mitigação Crítica): Adicionar circuit breaker nas chamadas às Google Directions API e Static Maps API no Backend e no worker, com fallback de ETA estimado por distância Haversine e geração de relatório sem imagem PNG quando a API estiver indisponível, preservando encerramento do passeio e dados textuais (distância, duração) mesmo sob throttling (R-02, S-04).
- Ação de Curto Prazo (Mitigação Crítica): Implementar reconexão automática com backoff no WebSocketGateway e nos clientes Flutter, com buffer local de coordenadas GPS no app do passeador para reenvio em caso de queda temporária de conexão, reduzindo perda de pontos da trilha (R-03, R-04, C-01, C-02).
- Ação de Curto Prazo (Mitigação Crítica): Padronizar permissões de localização em background no Flutter (Android: ACCESS_BACKGROUND_LOCATION; iOS: Always + UIBackgroundModes: location) e adicionar monitoramento de gaps na trilha com alerta ao tutor quando coordenadas deixarem de chegar por mais de 30 segundos (R-04, C-02).
- Ação de Médio/Longo Prazo: Migrar o RabbitMQ de instância single-instance para cluster Amazon MQ (active/standby ou multi-AZ), eliminando o ponto único de falha no broker e suportando picos de finalização simultânea (R-01, C-07, S-02).
- Ação de Médio/Longo Prazo: Extrair o WebSocketGateway do Backend NestJS para serviço dedicado (ou adotar Redis Pub/Sub como camada de fan-out), permitindo escalar horizontalmente conexões de rastreamento independentemente da API REST (R-03, T-03, S-03).
- Ação de Médio/Longo Prazo: Implementar testes de caos periódicos simulando indisponibilidade do worker, do RabbitMQ e das Google Maps APIs, validando retenção de mensagens, reconexão WebSocket e integridade dos dados no PostgreSQL após recuperação (R-01, R-02, NR-01).

- Ação de Médio/Longo Prazo: Avaliar cache de tiles de mapa e pré-cálculo de rotas frequentes para reduzir chamadas à Google Maps Platform, protegendo quota mensal e diminuindo latência de ETA com monitoramento de consumo e alertas em 80% do crédito (T-04, S-04, RNF010).
- Ação de Médio/Longo Prazo: Evoluir o modelo de persistência de coordenadas para escrita em batch no PostgreSQL (agrupamento a cada N pontos ou janela temporal), reduzindo contenção no RDS e carga no WebSocketGateway sem comprometer a latência percebida de 30 segundos (T-01, S-01, C-01, C-03).